

# PROJECT REPORT: CLOUDDEPLOY PaaS

To: Engineering Management  
Prepared By: CloudDeploy Engineering Team  
Subject: CloudDeploy Platform Implementation & Workflow Report

## 1. THE PROBLEM

Modern software development teams struggle with significant operational friction when moving application code from repository to production. The primary problems addressed by this project include:

- **High DevOps Overhead:** Software developers must write complex Dockerfiles, configure container registry uploads, and manage verbose Kubernetes YAML manifests (Deployments, Services, Ingress, HPAs, and Secrets). This diverts time away from writing core product features.
- **Slow Feedback Loops:** Traditional CI/CD setups are complex to build, modify, and monitor. Developers lack real-time visibility into builds, container pushes, and scheduling states.
- **Environment Configuration Bottlenecks:** Synchronizing environment variables and API keys across multiple cloud environments, while ensuring secrets are encrypted and rotated, is error-prone and insecure.
- **Underutilized Infrastructure:** Deploying applications to dedicated virtual machines results in low resource utilization and high cloud costs. Managing shared clusters manually introduces resource conflicts and security risks.

## 2. THE SOLUTION

CloudDeploy is a self-service Platform-as-a-Service (PaaS) built on Kubernetes. It abstracts away all cloud infrastructure complexity. Key solution pillars include:

- **Zero-YAML Deployments:** Developers only need to provide a GitHub repository URL. CloudDeploy automatically builds, containerizes, pushes, and schedules the application on a Kubernetes cluster.
- **Automated Kubernetes Orchestration:** The platform dynamically configures isolated namespaces, ClusterIP services, NGINX Ingress routing, and Horizontal Pod Autoscaling (HPA) without manual operator intervention.
- **Real-time Pipeline & Feedback:** Utilizing asynchronous workers and WebSockets, the dashboard streams live container build logs, deployment rollout states, and live pod health metrics to the developer.
- **Secure Config Management:** Provides a centralized dashboard interface to manage environment variables. These are automatically synchronized as native Kubernetes ConfigMaps and Secrets, triggering rolling updates automatically.

## 3. THE TECHNOLOGY STACK

| Layer            | Technology                            | Rationale                                                                                             |
|------------------|---------------------------------------|-------------------------------------------------------------------------------------------------------|
| Frontend UI      | React 18, Vite, Tailwind CSS, Zustand | Offers a highly responsive, modern interface with real-time state management and zero build overhead. |
| Backend API      | Node.js, Express, Socket.io           | Provides a lightweight, event-driven backend. Socket.io enables real-time duplex log streaming.       |
| Data Persistence | MongoDB 7, Mongoose                   | Facilitates flexible storage of user profiles, project configs, histories, and system                 |

logs.



Asynchronous Queue



Redis, BullMQ

Ensures reliable task serialization. Long-running builds run in background threads, avoiding API blockages.

Container Engine

Docker, dockerode

Programmatically clones git repositories, builds optimized Docker images, and securely pushes them.



Orchestration Client

Kubernetes  
(@kubernetes/node) SDK

Directly integrates with the Kubernetes API to programmatically provision and scale cluster resources.

Ingress & Networking

NGINX Ingress, cert-manager

Dynamically routes HTTP/HTTPS traffic using wildcard DNS and provisions Let's Encrypt TLS certificates.

## 4. THE DEPLOYMENT WORKFLOW

Each project deployment follows a structured, automated lifecycle. Below is the step-by-step pipeline executed when a build is triggered:

- 1. Developer Request:** The user links their GitHub repository URL and clicks 'Deploy' in the React UI, or pushes new code to their branch (which triggers an incoming GitHub Webhook).
- 2. Queueing (Reliability Layer):** The API server receives the request and adds a job containing repository credentials, the commit hash, and project configurations to a BullMQ Redis Queue.
- 3. Source Code Retrieval (GitHub):** The worker picks up the job, creates a unique directory under `./backend/repos/<project-slug>`, and clones the target branch.
- 4. Containerization (Docker):** The worker interacts with the local Docker daemon via the dockerode SDK to build the container image dynamically, tagging it with the incremented deployment build version (e.g., `v1.0.0`).
- 5. Publishing (Registry):** The worker authenticates with the configured container registry (e.g. Docker Hub, ECR) and pushes the newly built image.
- 6. Orchestration & Deploy (Kubernetes):** The system connects to the Kubernetes API and applies namespace templates: namespaces, environment config maps, deployments with rolling updates, stable clusterIP services, and ingress routes.
- 7. Execution & Health:** Kubernetes starts the pods, monitors Liveness/Readiness probes, and routes traffic. The application URL becomes live and accessible.